

Project 4 Report – Human-Centric Artificial Intelligence (HCAI)

Name – Surname: Barış Kaplan

Matriculation Number (Matrikelnummer): 669584

Task 1:

I have used a hybrid approach for the feature representation. It combines 2 components: **“Multi-Hot Encoded Movie Genres”** and **“Minimum-Maximum Scaled Numerical IMDB Rating Values”**. **“Multi-Hot Encoded Genres”** includes the categorical text data that represents the genres of the movie. I have chosen the features of movie genre and IMDB score, since they better represent the user’s movie selection/order and provide more realistic movie(s) to be recommended. If a movie is included under a category, the genre matrix takes the value 1 for the given movie and category. Otherwise, the genre matrix takes the value of 0. **“Minimum-Maximum Scaled Numerical Ratings”** includes continuous numerical data that represents the user and critic evaluations and feedback given to the movie by using the corresponding **‘imdb_score’** value of that movie.

Movies in the dataset usually do not belong to just a single type (e.g., Sci-Fi | Action | Adventure). For instance, the first movie entry located in the first row has a genres column with the value of **"Action|Adventure|Fantasy|Sci-Fi"**, which means that this movie is included in 4 movie genres, which are **"Action"**, **"Fantasy"**, **"Adventure"**, and **"Sci-Fi"**. In this dataset, each movie can belong to multiple genres (multi-label or categorical movie genres). Therefore, I have used a count vectorizer solution (**CountVectorizer(tokenizer=lambda text: text.split('|'), token_pattern=None, lowercase=True, binary=True)**), which is split by the pipe (|) parameter and applies a multi-hot encoding approach. The tokenizer parameter tells the CountVectorizer how to split each input string into separate tokens/labels. Since the movie genres for each movie in the dataset are separated by '|', I have split multiple genres per movie by this delimiter in the tokenizer part. The **'token_pattern'** parameter indicates the regular expression (regex) used by the CountVectorizer, which helps how to decide whether a token is encountered. I have set this parameter to None since I have not used the default regex but used my own custom regular expression. I have set the **'lowercase'** parameter to True to convert all processed letters to their lowercase versions, and this will count the same movie genres written in different cases as the same. I have set the **'binary'** parameter to True to just look for whether a word occurs or not

in the text input, and not count the word frequency. Because the multi-hot encoding approach includes just the binary values of 0 and 1, and not the frequency. This count vectorizer approach creates a multi-hot binary vector, where each row can contain multiple 1s. I could have also utilized a "Bag-of-Words (BoW)" approach for multi-hot encoding.

Using the count vectorizer or bag-of-words approach lets the movie recommender model capture the user's genre preferences (e.g., comedy, adventure, action) by calculating the inner product of the user's preference vector w and the movie feature vector x ($w^T * x$). Here, the utility for a given movie and user is the value of the inner product of vectors w and x .

The rating of a movie is treated as a reliable indicator of general movie appeal by audiences and critics, and of the overall movie quality. I have applied a scaling operation to every IMDb score in the dataset by using the `MinMaxScaler()` class. This normalization operation restricts the value of the IMDb score between 0 and 1, which is significant since it makes sure that the IMDb score does not dominate the binary movie genre dimensions while calculating the linear utility values using the $w^T * x$ multiplication, the dot product of the vectors w and x . Because, originally, in the given dataset, the IMDb score of a movie takes values between 0 and 10. Moreover, the binary movie genre value, depending on the existence of the movie in the specified genre, lies between 0 and 1. The linear utility model ($w^T * x$) will overemphasize the IMDb score over the movie genre if normalization to the range between 0 and 1 is not applied. Because the range of IMDb scores (**0-10**) is much higher than the binary movie genre values (**0-1, 0 or 1**). After the normalization, both the IMDb score and genre value will be in the range of 0 to 1, and thus equally emphasized, which leads to more realistic (fairer) utility scores of the linear model.

Task 2:

The Bradley-Terry model applies a Sigmoid (Logistic Regression) function to the difference of the utility values for the given vectors of movie preference of the user and movie features. I have used the gradient descent algorithm while updating the user's movie preference vector.

I have used the `plackett_luce_probability()` function for the extension to the ranking of a list of movies, with multiple (more than 2) movies involved rather than comparing just two movies. So, instead of the sigmoid function, I have applied the softmax function to each of the movies in the list and the remaining ones. The Plackett-Luce Probability Model extends the

Bradley-Terry Probability Model, and this extension can handle full rankings of items in the list with a size greater than 2. The Plackett-Luce model calculates the probability of an ordered list of choices through the decomposition into the multiplication of sequential Multinomial Logistic Regression (Softmax) function outputs; in other words, Softmax probabilities, given the k th movie item and the remaining movie items, which have indices starting from the number k and ending at the number of elements in the list, referred to as n . For the Plackett-Luce model, which is the extension of the Bradley-Terry model, the value of n should be greater than 2. Here, each of the multinomial softmax probabilities is the softmax function applied to the logit (raw utility score) of each choice, which represents its individual and comparative preference value.

In the extension, the probability of observing the ranked sequence $i_1, i_2, \dots, i_n$ is written as follows, given item raw utility scores (logits) calculated using $u_i = w^T x_i$:

$$P(i_1 > i_2 > \dots > i_n) = \prod_{m=1}^n \frac{e^{u_{i_m}}}{\sum_{j=m}^n e^{u_{i_j}}}$$

In this extended formula, the process of choosing the movie item is sequential. The item i_m is selected from the set of items $\{i_m, i_{m+1}, i_{m+2}, \dots, i_n\}$ with a probability that is directly proportional to the exponential of its utility, which is divided by the sum of exponentials of the utility values of all available movie items in the list.

When we plug $n=2$ into the formula above, the Plackett-Luce model (**multi-item ranking extension**) reduces to the Bradley-Terry model, which calculates the probability based on the pairwise item comparison. In both of the designs that I have implemented (Pairwise comparison and multi-item ranking), use the “**Gradient Descent**” algorithm to update and optimize the weights based on the sign of the gradient calculated.

When $n = 2$, for $m = 1$, the inner value in the product will become :

$$\frac{e^{u_{i_1}}}{e^{u_{i_1}} + e^{u_{i_2}}}$$

When $n = 2$, the last value of m will be 2. So, the inner value of the product will become:

$$\frac{e^{u_{i_2}}}{e^{u_{i_2}}} = 1$$

When we multiply them, we will eventually get the following:

$$P(i_1 > i_2) = \frac{e^{u_{i_1}}}{e^{u_{i_1}} + e^{u_{i_2}}} * \frac{e^{u_{i_2}}}{e^{u_{i_2}}} = \frac{e^{u_{i_1}}}{e^{u_{i_1}} + e^{u_{i_2}}} * 1 = \frac{e^{u_{i_1}}}{e^{u_{i_1}} + e^{u_{i_2}}}$$

$$P(i_1 > i_2) = \frac{e^{u_{i_1}}}{e^{u_{i_1}} + e^{u_{i_2}}}$$

So, at the end, this formula is equal to the formula that I have used in my first design implementation, which is the calculation of the probability of pairwise item comparison using the Bradley-Terry model.

Note: Please note that the full ranking of the set of ten movies is implemented in the second design, which operates on multi-item lists that contain more than 2 movie items. However, this can be reduced to partial ranking of multiple items, where we, for instance, rank top- k movies out of the n movies in total, where $0 < k < n$, from the most preferred movie to the least preferred one (For example: $k = 6$ and $n = 10$). In my implementation, I have used the ‘**top_n**’ parameter in the ‘*recommendations_view*’ function to decide how many movies will be recommended.

The Difference of the Learning Rates of Design 1 and Design 2

The learning rate of Design 1: 0.05

The learning rate of Design 2: 0.005

I have selected the learning rate of Design 1 to be greater than the learning rate of Design 2 because of the differences in information content extracted per movie recommendation vector update and the gradient accumulation amounts. Design 1 operates just on 1 pair of movie items, whereas Design 2 operates on a set of 10 movies. Therefore, this

means that Design 1 calculates the gradient value in just 1 iteration, while Design 2 calculates the gradient value by looping through 10 iterations and adding up 10 different difference vectors into the gradient vector to calculate the final gradient value. As a result, this makes the gradient calculated in Design 2 greater than the gradient calculated in Design 1. Moreover, a single step of the binary movie selection makes the gradient lower-valued, lower-density, and noisier. On the other hand, ranking 10 movie items from the most preferred to the least preferred one brings a higher-valued, higher-density, and less noisy gradient. Therefore, to stabilize the updates of the preference vector w (to avoid an overly large amount of update step) and prevent overfitting (**learning the existing movies a lot and not being able to (adequately) adapt to randomly and newly chosen movies from the dataset**) and overshooting (**divergence, significant increases, or significant fluctuations**), I have chosen the learning rate of Design 1 (**0.05**) higher than the learning rate of Design 2 (0.005), where the accumulated gradient is less than the Design 1.

Task 3:

The Research Hypothesis (The Quality of Movie Recommendation): The method used in Design 2 (Plackett-Luce model's ranking of a set of 10 movie items) results in faster convergence of the user preference vector w and higher-quality recommendations than the method used in Design 1 (Bradley-Terry model's pairwise selection). Because Design 2's method extracts more informational gain per user interaction.

The within-subjects design is used instead of the between-subjects experimental design. In this design methodology, every participant in the user study is exposed to both movie elicitation methods and user interfaces. User study participants are randomly separated into two different, oppositely balanced groups to prevent order and learning implications:

- **Group 1:** The user completes Design 1, which involves 5 rounds of pairwise movie choices. He/she views movie recommendations after completing the rounds of Design 1. Then, the user completes Design 2, which involves ranking ten movies from the most preferred movie to the least preferred one. After that, he/she views movie recommendations after completing the ranking of 10 movies in Design 2.
- **Group 2:** The user completes Design 2, which involves ranking a set of 10 movies from the most preferred movie to the least preferred one. He/she views movie recommendations after completing the ranking of 10 movies in Design 2. Then, the user

completes Design 1, which involves 5 rounds of pairwise movie selections. After that, he/she views movie recommendations after completing the rounds of Design 1.

Participant Screening and Recruitment:

1. Screening Evaluation Criteria: Firstly, participants in the user study must have basic digital skills to click on the preferred movie in the pairwise selection design (**Design 1**), drag and drop the movie items in the ranking of a set of multiple movie items (**Design 2**), and click on the buttons available in the user interfaces of the designs utilized, as well as in the user interface of the landing page. Moreover, these participants must regularly watch movies (at least 1-2 movies per week, as per possible requirements) and be accustomed to using movie streaming recommendation platforms.

2. Sample Type and Size: Due to the incomplete cognitive development of people under the age of 18, and possible legal and ethical problems that may arise from accepting them into the user study, the sample should involve adults and some elderly people as well. The age range of the sample can start at 18 and end at 55 (**18-55**). The **sample size** can be selected as **100**, which will provide enough statistical power for the analysis of the design of pairwise movie comparison and selection (**Design 1**). Because 500 rounds of data from 100 participants in the user study will provide enough information about movie preferences with more manageable round data, where each round only contains 2 movies, out of which 1 will be selected. Therefore, this will decrease the user's confusion more, compared to Design 2, where he/she needs to deal with the ranking of multiple movies at once. This mental clarity results in less varied and more consistent movie recommendations. Moreover, as the sample size increases, the movie recommendation results will be less random and more stable and reliable. The results will be closer to the true expected value. This proposition is known as the “**Law of Large Numbers**” in statistics. In addition, the preferences/rankings of the movies selected will approach a normal distribution as the sample size starts to increase.

3. Participant Outreaching and Recruiting Methods: The participants for our user study can be found through movie evaluation clubs (within mostly educational facilities such as universities and high schools), online movie forums, university mailing lists, and the databases of cinema venues. The participants should bring their “**National Identity Cards**” to prove that they are at least 18-years old and at most 55-years old.

Steps of the User Study Preference Practice:

1. **Introduction to the Study & User's Approval:** Participants in our user study land on a welcome page, review the details of the implementation choice of the extended model, read the user study design methodology, and submit informed user consent before starting the study.
2. **The First Movie Elicitation Method:** The participant interacts with the initially assigned elicitation method (e.g., comparing movie pairs or sorting a list of 10 movies via a drag-and-drop method from the most preferred movie to the least preferred one).
3. **Movie Recommendation Assessment:** The system redirects to *recommendations_view*, which displays the top-10 recommended movies based on the learned user's movie preference vector w . Participants rate the relevance of each movie recommendation by the method of Design 1 on a numerical scale that ranges from 1 to 10. The implementation can be changed to recommend more than 10 movies to have a more precise opinion of the users about the movie recommendations.
4. **The Second Movie Elicitation Method:** The participants complete the rating of each movie recommendation by the method of Design 1, and then rank the set of 10 movies from the most preferred movie to the least preferred one. After ranking, the participant should rate the trustworthiness of each movie recommendation by the method of Design 2 on a numerical scale that ranges from 1 to 10.
5. **Final Closing Interview:** A brief 5 to 10-minute qualitative summary interview should be conducted with the user study participants to capture user preferences about the perceived accuracy of the movie recommendations and intuitiveness of the user interfaces utilized in the study.
6. **The Plan of the Analysis of Movie Recommendation and User Feedback Data:**

This data can either be analyzed qualitatively or quantitatively:

- **Qualitative Data Evaluation:** Textual analysis of the transcripts of final closing interviews with the participants of the user study should be conducted to determine the pain points about the designs of the baseline and extended methods, the usability and

reliability of the user interfaces of both movie preference elicitation designs, and how well the movie recommendations reflect the expected movie preference of the user.

- **Quantitative Data Evaluation:** Average recommendation relevance ratings of each user study method in Design 1 and Design 2 should first be collected and saved. Then, they should be compared using hypothesis testing, which can be either Wilcoxon signed-rank tests (**Skewed data**) or paired t-tests (**Normally-distributed data**) depending on the distribution of the ratings of recommendation relevance. Moreover, the update magnitude of the movie recommendation vector per user interaction should be followed and analyzed to measure and evaluate the speed of algorithmic convergence of that vector, both per user interaction and in total.